

MODULE 5

Code Optimization - Principal sources of optimization, Machine dependent and machine independent optimizations, Local and global optimizations.

Code Generation - Issues in the design of a code generator, Target Language, A simple code generator.

CODE OPTIMIZATION

- The code produced by the straight forward compiling algorithms can often be made to run faster or take less space, or both. This improvement is achieved by program transformations that are traditionally called optimizations.
- Compilers that apply code-improving transformations are called optimizing compilers.
- Code optimization is an optional phase and it must not change the meaning of the program.
- Code optimization is achieved in 2 ways:
 - Redundancies in a program are eliminated
 - Computations in a program are rearranged or rewritten to make it execute efficiently

The criteria for code improvement transformations:

- The transformation must preserve the meaning of programs.
 - That is, the optimization must not change the output produced by a program for a given input, or cause an error such as division by zero, that was not present in the original source program.
- A transformation must, on the average, speed up programs by a measurable amount.
 - Reducing the size of compiled code is also an important factor.
 - Not every transformation succeeds in improving every program, occasionally an “optimization” may slow down a program slightly.
- The transformation must be worth the effort.
 - ie, the effort put on optimization must be worthy when compared with the improvement.

Organization for an Optimizing Compiler

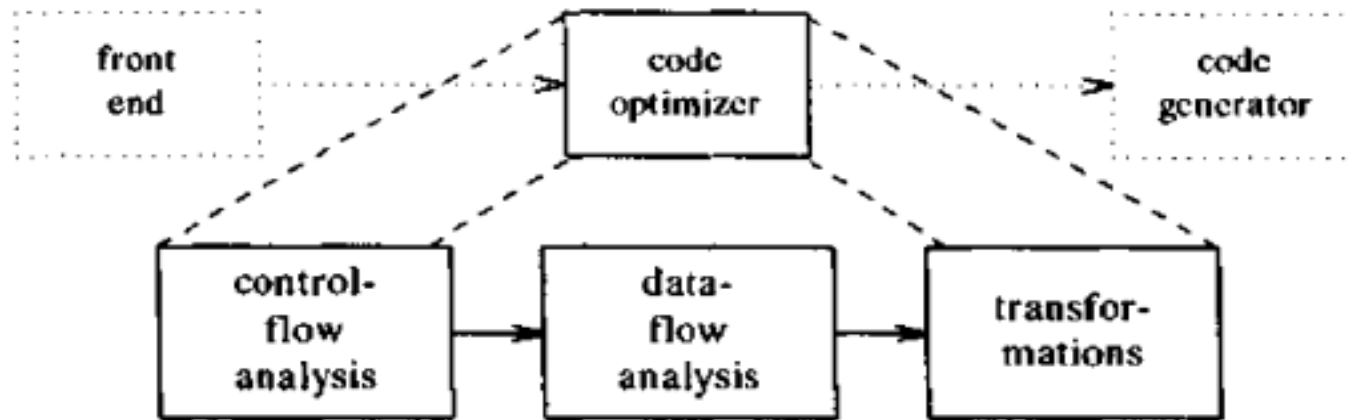


Fig. 10.3. Organization of the code optimizer.

- The code improvement phase consists of control-flow and data-flow analysis followed by the application of transformations.
- Control flow analysis (CFA) represents flow of control usually in form of graphs such as control flow graph and call graph.
- Data flow analysis (DFA) is the process of collecting information prior to program execution about the possible modification, preservation, and use of certain entities (such as values or attributes of variables) in a computer program.

- Optimizations are classified into two categories. They are
 - Machine independent optimizations.
 - Machine dependant optimizations.
- Machine independent optimizations:
 - Machine independent optimizations are program transformations that improve the target code without taking into consideration any properties of the target machine.
- Machine dependant optimizations:
 - Machine dependant optimizations are based on register allocation and utilization of special machine-instruction sequences.

Machine Dependent Code Optimization	Machine Independent Code Optimization
Machine dependent code optimization refers to optimizing code specifically for a particular type of computer or hardware platform	Machine independent code optimization refers to optimizing code for use on any type of hardware.
Machine dependent code optimization may require more maintenance and updates to keep the code optimized for different hardware platforms.	Machine independent code optimization requires fewer updates and is easier to maintain.
Machine dependent code optimization may take longer to develop, as it requires specialized knowledge of the hardware platform and may require testing on multiple platforms.	Machine independent code optimization can be developed more quickly.
Machine dependent code optimization may result in better performance on the specific hardware platform.	Machine independent code optimization may result in more consistent performance across all platforms.

- Optimization can be done in 2 phases:

1. Local optimization:

- Transformations are applied over a small segment of the program called basic block, in which statements are executed in sequential order.

2. Global optimization

- The global optimization allows the compiler/optimizer to look at the overall program and determine how best to apply the desired optimization level.
- Here transformations are applied over a large segment of the program like loop, procedures etc

Optimization of Basic Blocks

Basic Block

- A **basic block(BB)** is a sequence of program statements $(s_1, s_2, \dots, s_n)$ such that only s_n can be a transfer of control statement and only s_1 can be the destination of a transfer of control statement
- A BB is a program segment with single entry point.
- If control reaches statement s_1 during program execution, all statements $s_1, s_2 \dots s_n$ will be executed.
- The ‘essentially sequential’ nature of a BB simplifies optimization

Algorithm to determine Basic Blocks

1. Determine the set of leaders(1st stmt of the BB). The rules are follows
 1. 1st stmt of the program is a leader
 2. Any statement that is the target of a conditional or unconditional goto is a leader
 3. Any statement that is immediately follows a goto or unconditional goto is a leader
2. BB consist of a leader and all statements upto but not including the next leader or the end of the program

Example:

(1) $i := m - 1$	(16) $t_7 := 4 * i$
(2) $j := n$	(17) $t_8 := 4 * j$
(3) $t_1 := 4 * n$	(18) $t_9 := a[t_8]$
(4) $v := a[t_1]$	(19) $a[t_7] := t_9$
(5) $i := i + 1$	(20) $t_{10} := 4 * j$
(6) $t_2 := 4 * i$	(21) $a[t_{10}] := x$
(7) $t_3 := a[t_2]$	(22) goto (5)
(8) if $t_3 < v$ goto (5)	(23) $t_{11} := 4 * i$
(9) $j := j - 1$	(24) $x := a[t_{11}]$
(10) $t_4 := 4 * j$	(25) $t_{12} := 4 * i$
(11) $t_5 := a[t_4]$	(26) $t_{13} := 4 * n$
(12) if $t_5 > v$ goto (9)	(27) $t_{14} := a[t_{13}]$
(13) if $i \geq j$ goto (23)	(28) $a[t_{12}] := t_{14}$
(14) $t_6 := 4 * i$	(29) $t_{15} := 4 * n$
(15) $x := a[t_6]$	(30) $a[t_{15}] := x$

Fig. 10.4. Three-address code for fragment in Fig. 10.2.

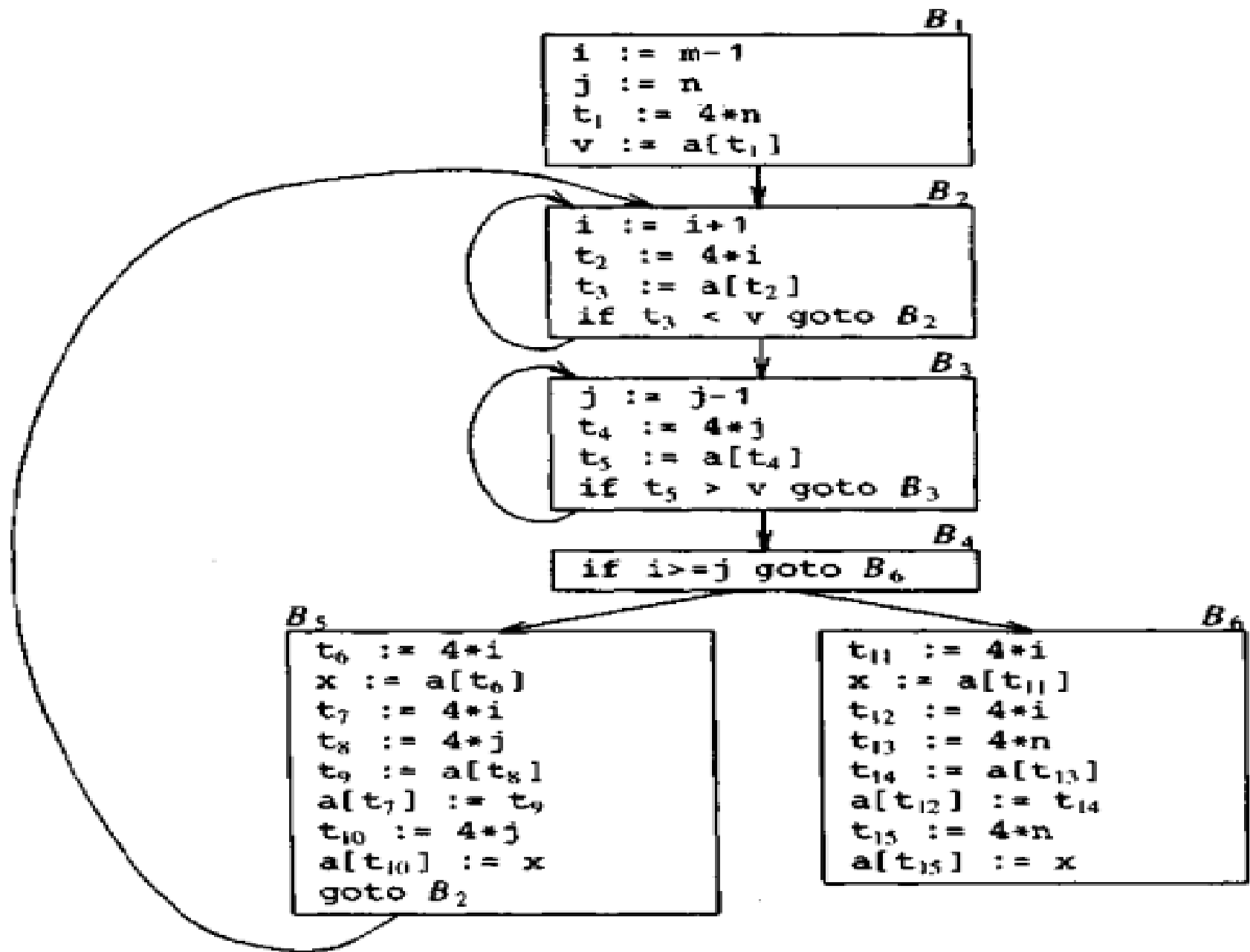


Fig. 10.5. Flow graph.

There are two types of basic block optimizations or transformations.

They are:

1. Structure-Preserving Transformations
2. Algebraic Transformations

1. Structure-Preserving Transformations:

- The primary Structure-Preserving Transformation on basic blocks are:
 - Common sub-expression elimination.
 - Dead code elimination.
 - Renaming of temporary variables
 - Interchange of two independent adjacent statements.

➤ Common sub-expression elimination:

Common sub expressions need not be computed over and over again. Instead they can be computed once and kept in store from where it's referenced when encountered again

B_5

```
t6 := 4*i  
x := a[t6]  
t7 := 4*i  
t8 := 4*j  
t9 := a[t8]  
a[t7] := t9  
t10 := 4*j  
a[t10] := x  
goto B2
```

(a) Before

B_5

```
t6 := 4*i  
x := a[t6]  
t8 := 4*j  
t9 := a[t8]  
a[t6] := t9  
a[t8] := x  
goto B2
```

(b) After

Fig. 10.6. Local common subexpression elimination.

➤ **Dead code elimination:**

It's possible that a large amount of dead (useless) code may exist in the program. This might be especially caused when introducing variables and procedures as part of construction or error-correction of a program – once declared and defined, one forgets to remove them in case they serve no purpose. Eliminating these will definitely optimize the code.

➤ **Renaming of temporary variables:**

- A statement $t := b + c$ where t is a temporary name can be changed to $u := b + c$ where u is another temporary name, and change all uses of t to u .
- In this we can transform a basic block to its equivalent block called normal-form block.

➤ **Interchange of two independent adjacent statements:**

- Two statements

$t_1 := b + c$

$t_2 := x + y$

can be interchanged or reordered in its computation in the basic block when value of t_1 does not affect the value of t_2 .

2. Algebraic Transformations

- Algebraic identities represent another important class of optimizations on basic blocks.

- Example:

$$x+0 = 0+x = x$$

$$x-0 = x$$

$$x*1 = 1*x = x$$

$$x/1 = x$$

- Another class of algebraic optimisation includes replacing an expensive operation by cheaper ones i.e. reduction in strength.

- Example:

$$x ** 2 = x*x$$

$$2.0*x = x+x$$

$$x/2 = x*0.5$$

- Another class of algebraic related optimizations is constant folding.
 - Here we evaluate constant expressions at compile time and replace the constant expressions by their values.
 - Thus the expression $2 * 3.14$ would be replaced by 6.28.

Principle Sources of Optimization

- A transformation of a program is called local if it can be performed by looking only at the statements in a basic block; otherwise, it is called global.
- Many transformations can be performed at both the local and global levels. Local transformations are usually performed first.

1. Function preserving transformations

- a. Common subexpression elimination
- b. Copy propagation
- c. Dead code elimination
- d. Constant folding

2. Loop optimization

- a. Code motion
- b. Induction variable elimination
- c. Reduction in strength

Function-Preserving Transformations

- There are a number of ways in which a compiler can improve a program without changing the function it computes.
- The common function preserving transformations are:
 1. Common subexpression elimination
 2. Copy propagation
 3. Dead-code elimination
 4. Constant folding
- The other transformations come up primarily when global optimizations are performed.

1. *Common Subexpression Elimination*

- An occurrence of an expression E is called a common sub-expression if E was previously computed, and the values of variables in E have not changed since the previous computation.
- We can avoid recomputing the expression if we can use the previously computed value.
- For example
t1: = 4*i
t2: = a [t1]
t3: = 4*j
t4:= 4*i
t5: = n
t6: = b [t4] +t5
- The above code can be optimized using the common sub-expression elimination as t1: = 4*i t2: = a [t1] t3: = 4*j t5: = n t6: = b [t1] +t5
- The common sub expression t4: =4*i is eliminated as its computation is already in t1. And value of i is not been changed from definition to use.

2. *Copy Propagation*

- Assignments of the form $f := g$ are called copy statements, or copies for short.
- The idea behind the copy-propagation transformation is to use g for f , whenever possible after the copy statement $f := g$.
- Copy propagation means use of one variable instead of another.
- This may not appear to be an improvement, but it gives us an opportunity to eliminate x .
- For example:
 $x = Pi;$

 $A = x * r * r;$
- The optimization using copy propagation can be done as follows:
 $A = Pi * r * r;$ Here the variable x is eliminated

3. Dead-Code Eliminations:

- A variable is live at a point in a program if its value can be used subsequently; otherwise, it is dead at that point.
- A related idea is dead or useless code, statements that compute values that never get used.
- While the programmer is unlikely to introduce any dead code intentionally, it may appear as the result of previous transformations.
- An optimization can be done by eliminating dead code.
- Example:

```
    i=0;  
    if(i==1)  { a=b+5; }
```
- Here, ‘if’ statement is dead code because this condition will never get satisfied.

4. Constant folding:

- If all operands are constants in an expression, then it can be evaluated at compile time itself. The result of the operation can replace the original evaluation in the program.
- This will improve the run time performance by avoiding evaluation at compile- time.
- For example,
 $a = 3.14157/2$ can be replaced by $a = 1.570$ there by eliminating a division operation.

Loop Optimizations:

- The running time of a program may be improved if we decrease the number of instructions in an inner loop, even if we increase the amount of code outside that loop.
- Three techniques are important for loop optimization:
 - A) Code Motion, which moves code outside a loop;
 - B) Induction-variable elimination, which we apply to replace variables from inner loop.
 - C) Reduction in Strength, which replaces expensive operation by a cheaper one, such as a multiplication by an addition.

A) Code Motion (or Frequency Reduction)

- An important modification that decreases the amount of code in a loop is code motion.
- A fragment of code that resides in the loop and computes the same value at each iteration is called a loop-invariant code. This code can be moved out of the loop by saving it to be computed only once, rather than with each iteration.
- For example, evaluation of `limit-2` is a loop-invariant computation in the following while-statement:

`while (i <= limit-2) /* statement does not change limit*/`

Code motion will result in the following

`t= limit-2;`

`while (i<=t) /* statement does not change limit or t */`

Execution time of a program can be reduced by moving code from a part of a program which is executed very frequently to another part of the program which is executed fewer times

		<code>x := 25*a;</code>
<code>for i := 1 to 100 do</code>		<code>for i := 1 to 100 do</code>
<code>begin</code>		<code>begin</code>
<code> z := i;</code>	$\Rightarrow$	<code> z := i;</code>
<code> x := 25*a;</code>		<code> y := x+z;</code>
<code> y := x+z;</code>		<code>end;</code>
<code>end;</code>		

Here `x := 25*a;` is loop invariant. Hence in the optimized program it is computed only once before entering the for loop. `y := x+z;` is not loop invariant. Hence it cannot be subjected to frequency reduction.

B) Induction Variables :

- A variable is called an induction variable if its value is altered within the loop by a loop-invariant value.

Example:

The code fragment below has three induction variables (i1, i2, and i3) that can be replaced with one induction variable, thus eliminating two induction variables.

```
int a[SIZE];
int b[SIZE];

void f (void)
{
    int i1, i2, i3;

    for (i1 = 0, i2 = 0, i3 = 0; i1 < SIZE; i1++)
        a[i2++] = b[i3++];
    return;
}
```

The code fragment below shows the loop after induction variable elimination.

```
int a[SIZE];
int b[SIZE];

void f (void)
{
    int i1;

    for (i1 = 0; i1 < SIZE; i1++)
        a[i1] = b[i1];
    return;
}
```

Notes:

Induction variable elimination can reduce the number of additions (or subtractions) in a loop, and improve both run-time performance and code space.

C) Reduction In Strength:

- Reduction in strength replaces expensive operations by equivalent cheaper ones on the target machine.
- Certain machine instructions are considerably cheaper than others and can often be used as special case of more expensive operators.
 - For example, x^2 is invariably cheaper to implement as $x*x$ than as a call to an exponentiation routine.
 - Fixed-point multiplication or division by a power of two is cheaper to implement as a shift.
 - Floating-point division by a constant can be implemented as multiplication by a constant, which may be cheaper.

Example: Replacement of a multiplication by an addition

- Here, high strength operator ‘*’ in ‘i*5’ occurring inside loop is replaced by a low strength operator ‘+’ in ‘itemp+5’

```
for i := 1 to 10 do
begin
    - - -
    k := i*5;
    - - -
end;
```

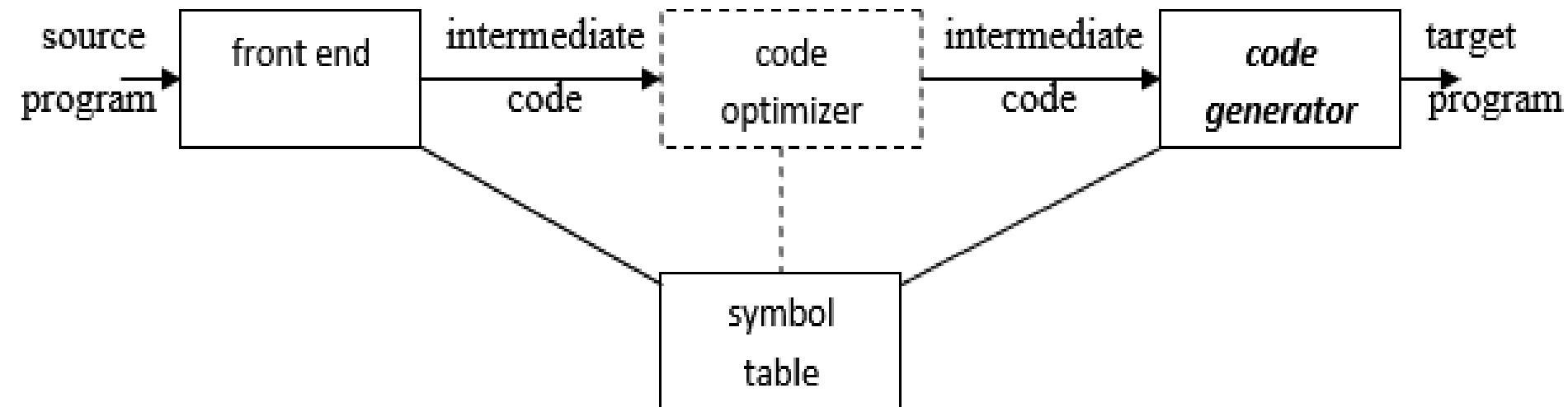
⇒

```
itemp := 5;
for i := 1 to 10 do
begin
    - - -
    k := itemp;
    - - -
    itemp := itemp+5;
end;
```

CODE GENERATION

- The final phase in compiler model is the code generator.
- It takes as input an intermediate representation of the source program and produces as output an equivalent target program.
- The code generation techniques can be used whether or not an optimizing phase occurs before code generation.

Position of code generator



Issues in the Design of Code Generator

- The following issues arise during the code generation phase :
 1. Input to code generator
 2. Target program
 3. Memory management
 4. Instruction selection
 5. Register allocation
 6. Evaluation order

1. Input to code generator:

- The input to the code generation consists of the intermediate representation of the source program produced by front end , together with information in the symbol table to determine run-time addresses of the data objects denoted by the names in the intermediate representation.
- Intermediate representation can be :
 - a. Linear representation such as postfix notation
 - b. Three address representation such as quadruples
 - c. Virtual machine representation such as stack machine code
 - d. Graphical representations such as syntax trees and dags.
- Prior to code generation, the front end must be scanned, parsed and translated into intermediate representation along with necessary type checking. Therefore, input to code generation is assumed to be error-free.

2. Target program:

- The output of the code generator is the target program. The output may be :
 - a. Absolute machine language
 - It can be placed in a fixed memory location and can be executed immediately.
 - b. Relocatable machine language
 - It allows subprograms to be compiled separately.
 - c. Assembly language
 - Code generation is made easier.

3. Memory management:

- Names in the source program are mapped to addresses of data objects in run-time memory by the front end and code generator.
- It makes use of symbol table, that is, a name in a three-address statement refers to a symbol-table entry for the name.
- Labels in three-address statements have to be converted to addresses of instructions.
For example,
 $j : \text{goto } i$ generates jump instruction as follows :
 - if $i < j$, a backward jump instruction with target address equal to location of code for quadruple i is generated.
 - if $i > j$, the jump is forward.

4. Instruction Selection

- The code generator must map the IR (Intermediate Representation) into a code sequence that can be executed by the target machine.
- The complexity of performing this mapping is determined by factors such as,
 - the level of the IR
 - the nature of the instruction-set architecture
 - the desired quality of the generated code
- If the IR is high level, the code generator may translate each IR statement into a sequence of machine instructions using code templates. Such statement-by-statement code generation often produces poor code that needs further optimization .
- The nature of the instruction set of the target machine has a strong effect on the difficulty of instruction selection. Uniformity and completeness of the instruction set are important factors.
- Instruction speed and machine idioms are other important factors. If we do not care about the efficiency of the target program, instruction selection is straightforward. For each common three-address statement, a general code can be designed.

For example, every three-address statement of the form $x := y + z$, where x , y , and z are statically allocated, can be translated into the code sequence

```
MOV y,R0    /* load y into register R0 */
ADD z,R0    /* add z to R0 */
MOV R0,x    /* store R0 into x */
```

Unfortunately, this kind of statement-by-statement code generation often produces poor code. For example, the sequence of statements

```
a := b + c
d := a + e
```

would be translated into

```
MOV b,R0
ADD c,R0
MOV R0,a
MOV a,R0
ADD e,R0
MOV R0,d
```

Here the fourth statement is redundant

5. Register Allocation

- ✚ A key problem in code generation is deciding what values to hold in what registers. Registers are the fastest computational unit on the target machine, but we usually not have enough of them to hold all values.
- ✚ The use of registers is often subdivided into two sub problems:
 - Register allocation, during which we select the set of variables that will reside in registers at each point in the program.
 - Register assignment, during which we pick the specific register that a variable will reside in.

6. Evaluation order

- The order in which the computations are performed can affect the efficiency of the target code.
- Some computation orders require fewer registers to hold intermediate results than others.

Target Machine

- Familiarity with the target machine and its instruction set is a prerequisite for designing a good code generator.
- The target computer is a byte-addressable machine with 4 bytes to a word.
- It has n general-purpose registers, $R0, R1, \dots, R_{n-1}$.
- It has two-address instructions of the form:
op source, destination where, op is an op-code, and $source$ and $destination$ are data fields.
- It has the following op-codes :
 - MOV (move source to destination)
 - ADD (add source to destination)
 - SUB (subtract source from destination)

- The source and destination of an instruction are specified by combining registers and memory locations with address modes. Here $\text{contents}(a)$ denotes the contents of the register or memory address represented by a .

Address modes with their assembly-language forms

MODE	FORM	ADDRESS	ADDED COST
<i>absolute</i>	M	M	1
<i>register</i>	R	R	0
<i>indexed</i>	$c(R)$	$c + \text{contents}(R)$	1
<i>indirect register</i>	*R	$\text{contents}(R)$	0
<i>indirect indexed</i>	* $c(R)$	$\text{contents}(c + \text{contents}(R))$	1
<i>literal</i>	# c	c	1

- For example : MOV R0, M stores contents of Register R0 into memory location M ;
- MOV 4(R0), M stores the value $\text{contents}(4 + \text{contents}(R0))$ into M.
- MOV *4(R0), M stores the value $\text{contents}(\text{contents}(4 + \text{contents}(R0)))$ into memory location M.
- MOV #1, R0 loads the constant 1 into register R0.

Instruction costs :

- Instruction cost = 1 + cost for source and destination address modes. This cost corresponds to the length of the instruction.
- Address modes involving registers have cost zero.
- Address modes involving memory location or literal have cost one.
- Instruction length should be minimized if space is important. Doing so also minimizes the time taken to fetch and perform the instruction.
- For example : MOV R0, R1 copies the contents of register R0 into R1. It has cost one, since it occupies only one word of memory.

- Store instruction MOV R5, M copies contents of R5 to Memory location M. This has cost 2, since address of Memory location M is in the word following instruction.
- The three-address statement $a := b + c$ can be implemented by many different instruction sequences :
 - i) MOV b, R0
ADD c, R0 cost = 6
MOV R0, a
 - ii) MOV b, a
ADD c, a cost = 6
 - iii) Assuming R0, R1 and R2 contain the addresses of a, b, and c :
MOV *R1, *R0
ADD *R2, *R0 cost = 2
- In order to generate good code for target machine, we must utilize its addressing capabilities efficiently

A Simple Code Generator

- A code generator generates target code for a sequence of three-address statements and effectively uses registers to store operands of the statements.
- For example: consider the three-address statement $a := b + c$. It can have the following sequence of codes:

ADD R_j, R_i Cost = 1 // if R_i contains b and R_j contains c

(or)

ADD c, R_i Cost = 2 // if c is in a memory location

(or)

MOV c, R_j Cost = 3 // move c from memory to R_j and add

ADD R_j, R_i

Register and Address Descriptors:

- A register descriptor is used to keep track of what is currently in each registers. The register descriptors show that initially all the registers are empty.
- An address descriptor stores the location where the current value of the name can be found at run time.

Code Generation Algorithm

- The algorithm takes as input a sequence of three-address statements constituting a basic block.
- For each three-address statement of the form $x := y \text{ op } z$, perform the following actions:
 1. Invoke a function **getreg** to determine the location L where the result of the computation $y \text{ op } z$ should be stored.
 2. Consult the address descriptor for y to determine y' , the current location of y . Prefer the register for y' if the value of y is currently both in memory and a register. If the value of y is not already in L , generate the instruction `MOV  $y'$ ,  $L$` to place a copy of y in L .

3. Generate the instruction OP z' , L where z' is a current location of z . Prefer a register to a memory location if z is in both. Update the address descriptor of x to indicate that x is in location L. If x is in L, update its descriptor and remove x from all other descriptors.
4. If the current values of y or z have no next uses, are not live on exit from the block, and are in registers, alter the register descriptor to indicate that, after execution of $x := y \text{ op } z$, those registers will no longer contain y or z .

Code generator uses *getReg* function to determine the status of available registers and the location of name values. *getReg* works as follows:

- If variable Y is already in register R , it uses that register.
- Else if some register R is available, it uses that register.
- Else if both the above options are not possible, it chooses a register that requires minimal number of load and store instructions. That is, it takes a register which is already occupied, move its contents into some memory using the instruction MOV R, M . and then uses the register R .

Generating Code for Assignment Statements:

- The assignment $d := (a-b) + (a-c) + (a-c)$ might be translated into the following three-address code sequence:

$t := a - b$

$u := a - c$

$v := t + u$

$d := v + u$

with d live at the end

Code sequence for the example is:

Statements	Code Generated	Register descriptor	Address descriptor
		Register empty	
$t := a - b$	MOV a, R0 SUB b, R0	R0 contains t	t in R0
$u := a - c$	MOV a, R1 SUB c, R1	R0 contains t R1 contains u	t in R0 u in R1
$v := t + u$	ADD R1, R0	R0 contains v R1 contains u	u in R1 v in R0
$d := v + u$	ADD R1, R0 MOV R0, d	R0 contains d	d in R0 d in R0 and memory

Generating Code for Indexed Assignments

The table shows the code sequences generated for the indexed assignment statements $a := b[i]$ and $a[i] := b$

STATEMENT	i IN REGISTER Ri		i IN MEMORY Mi	
	CODE	COST	CODE	COST
$a := b[i]$	MOV b(Ri), R	2	MOV Mi, R MOV b(R), R	4
$a[i] := b$	MOV b, a(Ri)	3	MOV Mi, R MOV b, a(R)	5

Generating Code for Pointer Assignments

The table shows the code sequences generated for the pointer assignments $a := *p$ and $*p := a$

STATEMENT	p IN REGISTER Rp		p IN MEMORY Mp	
	CODE	COST	CODE	COST
$a := *p$	MOV *Rp, a	2	MOV Mp, R MOV *R, R	3
$*p := a$	MOV a, *Rp	2	MOV Mp, R MOV a, *R	4

Generating Code for Conditional Statements

Statement	Code
if $x < y$ goto z	CMP x, y CJ< z /* jump to z if condition code is negative */
$x := y + z$ if $x < 0$ goto z	MOV y, R ₀ ADD z, R ₀ MOV R ₀ , x CJ< z

Machines implement conditional jumps in one of two ways. One way is to branch if the value of a designated register meets one of the six conditions: negative, zero, positive, nonnegative, nonzero, and nonpositive. On such a machine a three-address statement such as `if x < y goto z` can be implemented by subtracting `y` from `x` in register `R`, and then jumping to `z` if the value in register `R` is negative.

A second approach, common to many machines, uses a set of *condition codes* to indicate whether the last quantity computed or loaded into a register is negative, zero, or positive. Often a compare instruction (`CMP` in our machine) has the desirable property that it sets the condition code without actually computing a value. That is, `CMP x, y` sets the condition code to positive if $x > y$, and so on. A conditional-jump machine instruction makes the jump if a designated condition $<$, $=$, $>$, $\leq$, $\neq$, or $\geq$ is met. We use the instruction `CJ<= z` to mean "jump to `z` if the condition code is negative or zero." For example, `if x < y goto z` could be implemented by

```
CMP    x, y
CJ<    z
```

TUTORIAL 9

1. With suitable examples, explain loop optimization techniques.
2. With suitable example of a basic block, explain the code improving transformations of a basic block.
3. List the optimization techniques available in local and global optimization.

TUTORIAL 10

1. For the following C statement, write the three-address code and quadruples.

$S := A - B + C * D - E + F$

Also convert the three-address code into machine code.

2. Generate code sequence for the expression

$x = (a - b) + (a + c) + (a + c)$

3. Explain the getreg function